



Facultad de ingeniería

Docente

Leidy Mariani Dorado

Materia

Sistemas Distribuidos

Título

TurnoBank

Integrantes

Nasly Stefania Collazos

Popayán, Cauca 27 de Mayo de 2026

1. Descripción general del sistema

Propósito del sistema:

TurnoBank es un sistema distribuido diseñado para gestionar turnos en entidades bancarias. El sistema permite que los clientes del banco sean atendidos de forma ordenada y eficiente, asignando turnos consecutivos según el orden de llegada.

Problemática que resuelve:

En un banco tradicional, el manejo de turnos de forma manual genera desorden, largas esperas y mala experiencia para el cliente. TurnoBank automatiza este proceso, llevando un registro digital de usuarios, turnos asignados, notificaciones y servicios que ofrece el banco, todo de forma organizada y persistente.

Funcionalidades principales:

- Registro y consulta de usuarios del banco
- Asignación automática de turnos consecutivos (T1, T2, T3...)
- Registro automático de notificaciones cada vez que se crea un turno
- Gestión de los tipos de servicios bancarios disponibles
- Monitoreo del estado de todos los servicios en tiempo real
- Protección ante fallos mediante Circuit Breaker y recuperación Half-Open

Usuarios del sistema:

- Clientes del banco que solicitan un turno
- Personal del banco que administra los servicios
- Administradores que monitorean el estado del sistema

Alcance actual del sistema:

El sistema cuenta con 4 microservicios funcionales, un API Gateway como punto de entrada único, 4 bases de datos PostgreSQL independientes con persistencia mediante volúmenes Docker, y un sistema de monitoreo que incluye health checks, logs descriptivos, circuit breaker y recuperación automática.

2. Arquitectura Final del sistema

El sistema sigue una arquitectura de microservicios donde cada componente tiene una responsabilidad específica. El cliente nunca se comunica directamente con los servicios (todo pasa por el API Gateway).

Microservicios implementados:

Servicio	Puerto Interno	Puerto Expuesto	Descripción
gateway-service	5000	5000	Punto de entrada único. Maneja circuit breaker y monitoreo
users-service	5000	5001	Gestión de usuarios del banco
turns-service	5000	5002	Asignación de turnos consecutivos
notifications-service	5000	5003	Historial de notificaciones por turno
servicios-service	5000	5004	Tipos de servicios bancarios disponibles

Bases de datos:

Base de datos	Tecnología	Tablas
db-users	PostgreSQL 15	usuarios (id, nombre, email)
db-turns	PostgreSQL 15	turnos (id, turno, user_id)
db-notifications	PostgreSQL 15	notificaciones (id, user_id, turno, mensaje)
db-servicios	PostgreSQL 15	servicios (id, nombre, descripción)

Comunicación entre servicios:

La comunicación entre todos los componentes se realiza mediante HTTP/REST. El flujo principal al crear un turno es el siguiente:

- Cliente envía POST /turn al gateway (puerto 5000)
- Gateway redirige la petición a turns-service
- turns-service llama a users-service para validar que el usuario existe
- Si el usuario existe, turns-service crea el turno y lo guarda en db-turns
- turns-service llama a notifications-service para registrar la notificación
- El gateway devuelve la respuesta al cliente con el turno asignado

3. Justificación de la arquitectura

¿Por qué se dividió en microservicios?

Se decidió separar el sistema en microservicios porque cada funcionalidad tiene una responsabilidad diferente y puede cambiar o fallar de forma independiente. Si el servicio de notificaciones falla, los turnos siguen creándose. Si el servicio de servicios bancarios cae, los usuarios y turnos siguen funcionando sin ningún problema.

Responsabilidades de cada servicio:

- **gateway-service:** recibe todas las peticiones del exterior, aplica el circuit breaker, mide tiempos de respuesta y monitorea el estado de los demás servicios
- **users-service:** solo se encarga de crear y consultar usuarios. No sabe nada de turnos ni notificaciones
- **turns-service:** asigna turnos consecutivos. Valida que el usuario exista antes de crear el turno y avisa al servicio de notificaciones
- **notifications-service:** guarda el historial de todas las notificaciones enviadas a los usuarios
- **servicios-service:** administra los tipos de servicios que ofrece el banco (apertura de cuenta, crédito, etc.)

Ventajas obtenidas:

- Si un servicio falla, los demás siguen funcionando de forma independiente
- Cada servicio tiene su propia base de datos, por lo que los datos están aislados
- Es más fácil entender y modificar el código de cada servicio por separado
- El sistema puede recuperarse automáticamente gracias al circuit breaker y half-open

Dificultades encontradas:

- Configurar la comunicación interna entre contenedores usando los nombres del docker-compose
- Manejar el tiempo de arranque de PostgreSQL, se solucionó con reintentos automáticos en cada servicio
- Implementar correctamente el estado half-open para que solo deje pasar una petición de prueba sin abrir todo el tráfico

4. Tolerancia a fallos

Circuit breaker

Se implementó el patrón Circuit Breaker en el gateway-service con un circuito independiente por cada servicio. Esto significa que si el servicio de turnos falla, el circuito de turnos se abre pero el circuito de usuarios sigue cerrado y funcionando normal.

- Límite de fallos: 3 intentos fallidos consecutivos abren el circuito
- Cuando el circuito está abierto, el gateway responde con error 503 de forma inmediata sin intentar conectarse

Esto evita que el sistema gaste tiempo y recursos esperando conexiones que no van a funcionar

Recuperación Half-Open

Después de 10 segundos con el circuito abierto, el sistema entra en estado Half-Open. En este estado deja pasar una sola petición de prueba para verificar si el servicio se recuperó.

- **Si la petición de prueba funciona:** el circuito se cierra y el sistema vuelve a funcionar normal
- **Si la petición de prueba falla:** el circuito vuelve a abrirse y reinicia el temporizador de 10 segundos

Logs descriptivos

Cada servicio genera mensajes en los logs que permiten saber exactamente qué está pasando en el sistema. Por ejemplo:

- [USERS] Creando usuario: Ana García (**cuando se crea un usuario**)
- [TURNS] Turno generado: T1 (**cuando se asigna un turno**)
- [NOTIFICATIONS] Notificación guardada (**cuando se registra una notificación**)
- [INFO] Tiempo de respuesta turns POST: 0.07s (**tiempo que tardó la petición**)
- Fallo turns #1 / Circuito turns ABIERTO (**cuando el circuit breaker actúa**)

Health checks y Monitoreo

Cada servicio expone un endpoint /health que devuelve su estado actual. El gateway tiene dos niveles de monitoreo:

- /estado/users, /estado/turns, /estado/notifications, /estado/servicios **(consulta individual de cada servicio)**
- /monitor **(consulta todos los servicios a la vez y muestra estado, latencia y contadores de fallos y peticiones)**

5. Seguridad básica

Variables de entorno

Todas las credenciales del sistema se manejan mediante variables de entorno definidas en un archivo `.env` que no se sube al repositorio. En el repositorio solo existe un archivo `.env.example` con los nombres de las variables y valores de ejemplo, sin credenciales reales

Variable	Descripción
DB_USER	Usuario de la base de datos
DB_PASSWORD	Contraseña de la base de datos
DB_HOST_USERS	Host de la base de datos de usuarios
DB_HOST_TURNS	Host de la base de datos de turnos
DB_HOST_NOTIFICATIONS	Host de la base de datos de notificaciones
DB_HOST_SERVICIOS	Host de la base de datos de servicios

En el código, estas variables se leen con `os.getenv()` de Python, nunca están escritas directamente en el código fuente. Esto evita que las credenciales queden expuestas en el repositorio de GitHub.

6. Endpoints del sistema

Método	Endpoint	Descripción
GET	/users	Lista todos los usuarios
POST	/users	Crea un nuevo usuario
GET	/turns	Lista todos los turnos
POST	/turn	Crea un turno para un usuario
GET	/notifications	Lista todas las notificaciones
POST	/notify	Registra una notificación
GET	/servicios	Lista los tipos de servicios bancarios
POST	/servicios	Registra un tipo de servicio
GET	/resumen	Devuelve datos de todos los servicios juntos
GET	/monitor	Estado, latencia, fallos y peticiones por servicio
GET	/estado/users	Health check de users-service
GET	/estado/turns	Health check de turns-service
GET	/estado/notifications	Health check de notifications-service
GET	/estado/servicios	Health check de servicios-service

7. Tecnologías utilizadas

Tecnología	Uso
Python 3.11	Lenguaje de programación de todos los servicios
Flask	Framework web para crear los endpoints REST
PostgreSQL 15	Base de datos relacional, una por servicio
psycopg2	Librería Python para conectarse a PostgreSQL
Docker	Contenerización de cada microservicio
Docker Compose	Orquestación de todos los contenedores
Postman	Pruebas de los endpoints
GitHub	Control de versiones y repositorio del proyecto
draw.io	Diseño del diagrama de arquitectura